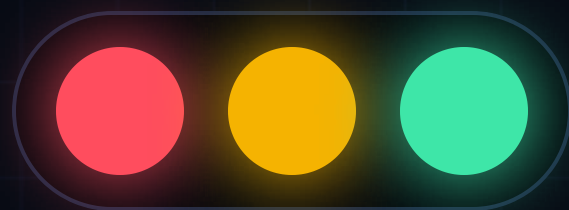


IBM BOB 2.0 HACKATHON

SIGNALBOX

Interlocking for parallel IBM Bob subagents.

Many agents, one codebase, no collisions. Every change is proven before it's committed.



THE PROBLEM

Parallel AI agents don't trust each other. Neither do we.

They collide

Two agents edit the same file.
One overwrites the other.

They break contracts

One agent renames a function
another agent's code still calls.

They cheat

A failing test gets "fixed" by
editing the test.

The result is one giant diff nobody can review. So teams run one agent at a time, and big migrations stay slow.

THE INSIGHT

Railways solved "many trains, one network" 150 years ago.



Block

A section of track. One train at a time.

→ the files one subagent owns



Signal

No entry until the line ahead is clear.

→ a wave opens when earlier waves clear



Track circuit

Proves the block is empty before release.

→ four checks before any commit

This is **interlocking**. Signalbox brings it to AI agents.

HOW IT WORKS

Bob plans, Bob works in parallel, the signal box keeps it safe.

1 Scan

chainguard finds 72 legacy call sites and reads the import graph

2 Plan

6 blocks in 2 waves. Blocks in a wave never touch each other

3 Dispatch

A Bob dispatcher agent starts a subagent for every green block

4 Claim

Each subagent claims its block. Everyone else is locked out

5 Prove

Release runs the track circuit on an isolated copy

6 Commit

A clear block is committed alone and signed by its agent. Otherwise, fix or roll back

No block is released until four checks pass.

S

Scope

Only this block's files changed. Anything else is a **SPAD** (signal passed at danger).

C

Contract

No export another file still imports was removed or renamed.

L

Legacy scan

Zero old-library calls left in the block.

T

Isolated tests

Run on a git worktree with only cleared work plus this block, so agents can't cause or mask each other's failures.

IBM BOB DOES THE WORK

Every capability the challenge names, doing real work.

Full-repo context + document understanding

Bob reads the repo and the migration playbook before touching anything

Agent mode, multi-step orchestration

A dispatcher agent reads the signal box, starts subagents, handles faults, moves wave by wave

Subagents + parallel tasks

One subagent per block, a whole wave at once, on files guaranteed not to overlap

Self-correction

A red track circuit hands Bob the failing test. Bob fixes the code, not the test

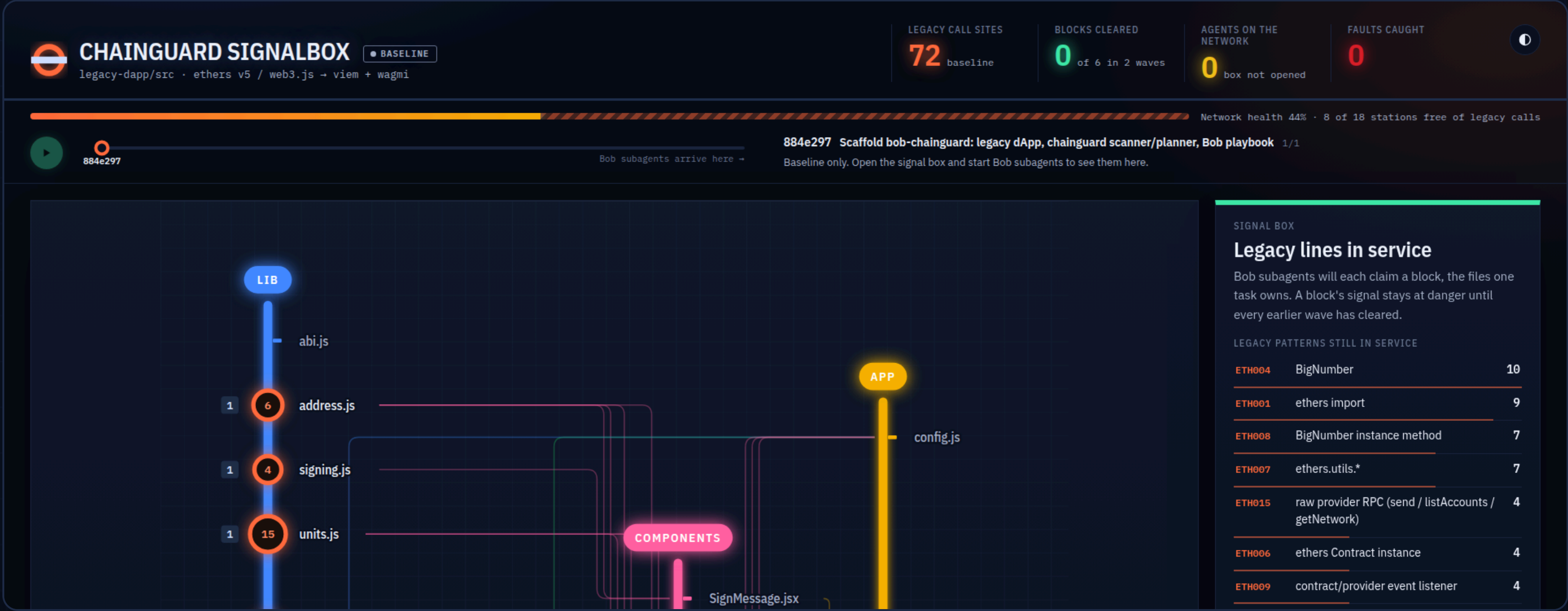
Code review

Bob reviews the full diff using the ledger of who changed what

SIGNALBOX

THE LIVE PANEL

Watch the agents work. Replay every decision.



Files are stations, folders are lines, imports are tunnels. Blocks light up as agents enter, faults flash red with the failing test, and the train graph shows the agents running in parallel.

SIGNALBOX

RESULTS

Every number is computed from the run.

72 → ?

legacy call sites

0/6

blocks cleared by Bob

?

Bob subagents

?

faults caught before commit

?

SPADs / rollbacks

?

wall clock

Run pending: **after the Bob run.** `npm run finalize` fills this slide from the ledger.

BUILT TO BE TRUSTED

Safe even when an agent misbehaves. Useful beyond this demo.

Safety net

- Tests and the checker are protected: agents can't edit them
- Live SPAD alarm the second an unclaimed file changes
- Pre-commit guard: no bypassing the signal box
- Black-box recorder restores work a rogue `git checkout` would wipe
- Expand → migrate → contract: shared code changes last, and only when unused

Any migration

- Rule packs: Web3 (ethers/web3.js → viem) built in
- Moment.js → date-fns pack included, 13 rules + playbook
- Same interlocking for framework upgrades, API renames, library swaps
- One command after the run: `npm run finalize`



Parallel Bob subagents you can actually trust.

github.com/ramadan904/bob-chainguard · ramadan904.github.io/bob-chainguard